

Q1)

A constructor in Java is a special kind of method that is invoked automatically when an instance of a class is created. The primary function of a constructor is to prepare the newly created object by assigning initial values to its attributes. Technically, a constructor is identified by the name that it shares with the class it belongs to and its characteristic of not having any return type, not even void. Typically, constructors work as facilitators in programming by making the process of creation of objects seamless and at the same time ensuring that the objects so created are validly initialized.

There are various types of constructors in Java and they are described like this. 1. Default Constructor

Java compiler automatically inserts a default constructor into programmed class if there are no existing constructors. This kind of constructor mainly sets the fields of the object with default values corresponding to their types such as 0, false or null. Example

```
class Student{ }
```

```
public class Demo{ public static void main(String[] args){
```

```
Student s = new Student(); }
```

```
} 2. No-Argument Constructor
```

Such constructors are written by the programmer and simply do not have any formal parameters. The object constructed through no-arg constructors are usually initialized to the default values stored inside the fields. Example

```
class Student{ Student(){
```

```
System.out.println("Student Object Created"); }
```

```
}' public class Demo{
```

```
public static void main(String[] args){ Student s = new Student();
```

```
}}
```

3. Parameterized Constructor By providing parameters, different initial values can be passed to different objects with the help of parameterized constructors.

Example class Student{

```
int id; String name;
```

```
Student(int i, String n){ id = i;
```

```
name = n; }
```

```
void display(){ System.out.println(id + " " + name);
```

```
}}
```

```
public class Demo{ public static void main(String[] args){
```

```
Student s = new Student(101, "Rahul"); s.display();
```

```
}}
```

4. Copy Constructor In Java, there is no ready-made copy constructor as in other languages, but we can define one by providing a constructor that takes an object of the same class as an argument.

Example class Student{

```
int id; Student(int i){
```

```
id = i; }
```

```
Student(Student s){ id = s.id;
```

```
}}
```

Benefits of Constructor It instantiates the object automatically.

Code duplication is minimized. Understanding the code is further aided.

Programming structure which is based on objects is encouraged. After all, the takeaway point about constructors is that they trigger the initialization of objects in Java at the moment when a new instance of the class is brought into existence. There are mainly the four types of constructors, that is default no-argument, parameterized, and copy constructors. And it is the degree of dependence on the source object for initialization that determines their respective usage scenarios and Because of this their utility.

Q2)

Control statements allow a programmer to alter the flow of execution in a Java program. They decide which code segments are to be executed, repeated, or omitted as certain conditions. With control statements, programming becomes more flexible and helps in simplifying decision-making.

Major types of control statements include:

1. Selection Statements

These statements run different sections of code based on a condition.

Examples:

if

if-else

if-else-if

switch

You use these when your program needs to select from multiple options.

2. Iteration Statements

These execute a particular section of code again and again until the condition you specify becomes false.

Examples:

for

while

do-while

This is a great way of avoiding writing the same code multiple times.

3. Jump Statements

Jump statements relocate the control from one region of the program to another.

Examples:

break

continue

return

These are typically used in the loops and methods.

Java Program to Calculate the Sum of $1 + 3 + 5 + \dots$ for 10 Terms

```
public class SeriesSum {  
  
    public static void main(String[] args) {  
  
        int sum = 0;  
  
        int num = 1;  
  
        for(int i = 1; i <= 10; i++){  
  
            sum = sum + num;  
  
            num = num + 2;  
  
        }  
  
        System.out.println("Sum = " + sum);  
  
    }  
  
}
```

Output

Sum = 100

Explanation

This Java application starts the odd numbers sequence from 1. A for loop executes the instructions within its body 10 times. At each iteration, the sum is increased by the present odd number and then the succeeding odd number is derived by adding 2.

Advantages of Control Statements

Enhance program's logical structure.

Minimize repeating code segments.

Facilitate decision-making.

Lead to more efficient programming.

To sum it up, control statements are very useful in Java programming. Selection statements help with making decisions, iteration statements allow repeated executions, and jump statements modify the regular execution flow. Individually or collectively, they make Java programs much more versatile and efficient.

Q3)

Filter streams are enhanced input and output streams in Java that offer additional functionalities over the basic byte and character streams. They do not interact with the file to read or write data physically. Actually, they get connected to another stream and furnish feature like buffering, data conversion and object serialization. Filter streams are part of the java.io package and matter a lot in enhancing input and output operations.

The essential function of filter streams is to simplify data processing and also to make it quicker and more efficient. They don't directly deal with the file but instead, they encapsulate another stream through which data processing is done. This way they can offer extra functionalities before the actual reading or writing of data happens.

Some of the popular filter stream classes are described below.

1. BufferedInputStream

This class reads data by employing an internal buffer. Buffering is known to cut down disk accesses and So improve reading speed.

2. BufferedOutputStream

It temporarily keeps output data in a buffer before writing it to the file. As a result, writing performance is enhanced.

3. DataInputStream

This class enables reading of primitive data types like int float double, and boolean from an input stream.

4. DataOutputStream

This class writes primitive data types in their binary forms which ease data storing and retrieval.

5. ObjectInputStream

It is a stream class that makes it possible to read objects from a file. It is designed for supporting object deserialization.

6. ObjectOutputStream

This is a stream class that writes Java objects into a file. This activity is known as object serialization.

Sample

```
import java.io.*;
```

```
public class Demo {
```

```
    public static void main(String[] args) throws Exception {
```

```
        FileOutputStream fos = new FileOutputStream("data.txt");
```

```
        BufferedOutputStream bos = new BufferedOutputStream(fos);
```

```
String msg = "Hello Java";

bos.write(msg.getBytes());

bos.close();

System.out.println("Data Written Successfully");

}

}
```

Benefits of Filter Streams

More rapid input and output operations.

Less frequent disk access.

Capable of serializing an object.

Simplifies the reading and writing of primitive data types.

Boosts the performance of the program.

To sum up, by employing filter streams, the basic input and output streams can be augmented by introducing buffering, object handling, and support for primitive data types.

Some of the classes like `BufferedInputStream`, `BufferedOutputStream`, `DataInputStream`, `DataOutputStream`, `ObjectInputStream`, and `ObjectOutputStream` are extensively employed in Java programs to handle files and data processing efficiently.

Q4)

Both interface and abstract class are used in Java for abstraction but differ in their usage. An abstract class allows partial implementation whereas an interface sets a contract that the implementation classes are bound to follow.

Comparison between Interface and Abstract Class

Interface	Abstract Class
Created with the interface keyword.	Created with the abstract keyword.
Allows pure abstraction only (default and static methods can also be included). Supports abstract as well as concrete methods.	
A class can implement multiple interfaces. A class can extend only one abstract class.	
Variables, by default, are public, static, and final. Can contain instance variables and constructors.	
Used when different classes need the same behavior. Used when classes share common features and code.	

Comparison between Errors and Exceptions

Both errors and exceptions are situations that disrupt the normal flow of the program. How they are dealt with But is different.

Errors Exceptions

They are very severe problems thrown by the JVM. They are faults that happen during the running of the program.

You, as a programmer, are not normally expected to handle these errors. You can deal with these through try-catch blocks.

They may happen due to lack of memory or system failure. They can be caused by invalid input, file not found, divide by zero, etc.

They are part of the Error class hierarchy. They are part of the Exception class hierarchy.

Example of Exception

```
try{
```

```
int a = 10 / 0;
```

```
}
```

```
catch(ArithmeticException e){
```

```
System.out.println("Cannot divide by zero");  
  
}
```

In this case, the exception is caught and processed, thereby the program is not terminated.

Advantages of Interface

It enables multiple inheritance.

It offers full abstraction.

It increases the possibilities of flexibility and reusability of the code.

Advantages of Abstract Class

You can use existing code again.

It allows you to have both abstract and concrete methods.

This is the right choice when the classes are related and they need to share the same functionalities.

So, interfaces and abstract classes are key components of Java. Mainly interfaces are used to define uniform behavior among classes that do not have any relationship, while abstract classes come handy when classes have common attributes and methods. Also, errors

stand for very serious system breakdowns whereas exceptions are program problems that developers can resolve through the use of exception handling ways.

Q5)

JDBC, which stands for Java Database Connectivity, is a Java API that lets developers connect their Java programs to relational databases such as MySQL Oracle PostgreSQL, and SQL Server. It defines the way a developer can send SQL queries, fetch records, and update databases. In essence, JDBC bridges the gap between Java programs and database management systems.

Summary of components in a JDBC application architecture:

1. Java Program

The Java program consists of business logic. JDBC API is the set of tools that are used in that program to access the database.

2. JDBC API

Database operations are performed using the classes and interfaces that JDBC API has to offer, like Connection Statement PreparedStatement ResultSet DriverManager.

3. JDBC Driver

JDBC driver is what interprets Java calls to database commands specific to the given database. Database vendors provide their own specific JDBC drivers.

4. DriverManager

DriverManager is responsible for loading the correct JDBC driver and initiating the communication between the Java program and the database.

5. Database

Data tables indexes, etc. -- all the artifacts of database are kept here. JDBC is used to run SQL statements on the database.

JDBC flow

JDBC flow at the simplest level is ::

Load the JDBC driver.

Make a database connection.

Prepare a Statement or PreparedStatement object.

Run SQL queries.

Handle the ResultSet.

Disconnect from the database.

Merits of JDBC

Easy database connectivity.

Works with many database servers.

Allows the use of PreparedStatement for making data access more secure.

Running SQL statements is very easy.

Works anywhere.

Instances

Banking systems.

Student management systems.

Online shopping systems.

Hospital management systems.

Inventory management.

Basically, JDBC is a structure for connecting Java-based programs and databases. Its primary components are the Java program, JDBC API, JDBC driver, DriverManager, and database. The five elements jointly make it feasible and safe to work with databases.

Q6)

Inheritance is arguably the most significant aspect of Java's Object-Oriented way of thinking. It enables a class to utilize the properties and methods of another class.

The existing class is referred to as the parent class (superclass) and the new class is known as the child class (subclass).

In Java it is implemented by the use of the extends keyword.

Inheritance, There are two benefits to inheritance, code sharing and method overriding.

Code sharing means that the class utilizing inheritance can use all the members of inherited class without re-writing the same code.

Method overriding is defining a way for the child class to alter the functionality of methods inherited from the parent class.

```
Class Parent{ 15: public static void main(String[] args){ 16: Parent p= new Parent(); 17: Child c= new Child(p); 18: Production.
```

```
()13 19:;}} When you run this code and the output shows Production.
```

Value=138, then you know that Child got correct Parent instance when getting executed.

Then you are sure that the execution order is correct. `Void display(){ System.out.`

8}. } thereby we minimize the chance of, it being too sensitive to the liberalization of the Thai economy.

For CG, it only relies on two domestic variables; consumer expenditures and the real interest rate.

Because of this, it may be sensitive to shocks in Thai economy. Child class Child extends Parent} A simple example: `System.out.`

It is prohibitively expensive to simply read, but it becomes feasible once the index has been built, by only the extra cost related to locating and fetching the relevant page.

So, the more the simulated applications make use of the search engine, the less overhead there will be.

This is obviously, countered by the time required to construct and maintain the index.

```
} Public class Demo{ Static public void main(String[]args) Child obj = new Child(); //obj.
```

`display(); Obj.show();` Can be quite complex.

For example, the so-called complex minimum between the two mixture components (Figure 9) is a convex region.

If merged, the remaining two component distributions would exert a local minimum between themselves a convex shape shown as free lines in the figure.

1. Single Inheritance

There is only one child class and it gets inherited from one parent class. 2. Multilevel Inheritance

A class orders its inherits from other child classes, developing some inheritance levels. 3. Hierarchical Inheritance

A number of children classes all derive from the same parent class.

Multiple inheritance is not supported for classes in Java because it will not support by the language and can bring confusion.

Multiple inheritance is achievable using interfaces in the language.

? Encourages code reuse. Reducing code repetition. Supports method overriding. 201. Makes programs easier to support. Largely reflects real-world relationships. Applications

Systems of banking. Employee management systems. Library management application. School Management applications. E-commerce system.

To wrap up, inheritance is one of the most powerful feature of object-oriented programming. It allows us to inherit the attributes and methods of one class to another class.

It supports inheritance types such as single inheritance, multilevel inheritance, and hierarchical inheritance using extends.

It makes easier code reusability, easy maintainability, and makes our development process organized and scalable.